

❏ 1. SAST and DAST Tools

❏ SAST (Static Application Security Testing)

Definition:

SAST is a white-box testing method that analyzes source code, bytecode, or binary code **at rest**, without executing the program.

Key Features:

- Performed early in the software development lifecycle (SDLC).
- Helps detect vulnerabilities like input validation issues, buffer overflows, hardcoded secrets, etc.
- Fast feedback for developers.

Examples of SAST Tools:

- SonarQube
- Fortify Static Code Analyzer
- Checkmarx
- Veracode Static Analysis

Advantages:

-

Identifies security vulnerabilities early in the development process.

- Can be integrated into CI/CD pipelines for real-time feedback.
- Helps enforce secure coding practices.

Limitations:

- Might produce **false positives**.
- Requires access to source code.
- Doesn't detect runtime or environment-specific issues.

☒ DAST (Dynamic Application Security Testing)

Definition:

DAST is a black-box testing method that analyzes applications **during execution** to find security vulnerabilities.

Key Features:

- Tests the application from the outside in (no code access required).
- Performed in staging or production-like environments.
- Detects vulnerabilities like XSS, SQL injection, CSRF, etc.

Examples of DAST Tools:

- OWASP ZAP
- Burp Suite
- Acunetix
- AppScan

Advantages:

- Tests the actual deployed application.
- Effective at finding runtime vulnerabilities.
- Can be used even if the source code is not available.

Limitations:

- Limited in identifying logic flaws or code-level issues.
- Might not cover all app behavior, especially under complex user conditions.
- Typically performed later in SDLC, which can delay fixes.

❏ SAST vs DAST – Comparison Table:

Feature	SAST	DAST
Code Access	Requires source code	No code access needed
Execution Required	No	Yes
Testing Phase	Early in SDLC	After deployment
Detects	Code-level flaws	Runtime vulnerabilities
False Positives	Can be high	Usually lower
Integration	CI/CD pipelines	QA or staging environments

❏ 2. Case Study: Web Application Frameworks

What Is a Web Application Framework?

A web framework provides a structure and reusable components to build web applications faster and more securely.

Common Web Frameworks:

- Django (Python)
- Ruby on Rails (Ruby)
- Laravel (PHP)

- ASP.NET (C#)
- Spring Boot (Java)
- Express.js (Node.js)

Security Features Typically Provided:

- Input validation and sanitation
- Built-in user authentication and authorization
- ORM (Object-Relational Mapping) to prevent SQL injection
- CSRF and XSS protection
- Secure session and cookie management

Case Study: Django Framework

Strengths:

- Middleware for security (XSS, CSRF, Clickjacking protection)
- Secure password storage (PBKDF2 hashing by default)

- Form validation and data sanitization

Common Pitfalls:

- Misconfigured settings (e.g., DEBUG=True in production)
- Over-reliance on built-in features without understanding
- Lack of custom access control checks in views

Lessons Learned:

- Developers must review and configure security settings manually.
- Frameworks provide tools, but secure usage is the developer's responsibility.

☒ 3. Using browser-jsguard Firefox Add-On

Overview:

browser-jsguard is a Firefox extension that detects:

- Malicious JavaScript behavior
- Suspicious web activity like phishing or hidden scripts

- Unexpected iframe or popup generation

Key Features:

- Real-time detection of malicious scripts based on heuristic patterns.
- Warns users when a website tries to execute suspicious JavaScript code.
- Logs browser behaviors and script calls that deviate from safe baselines.

Use Cases:

- Detect crypto-miners embedded in websites.
- Identify drive-by-download attempts.
- Monitor web pages for obfuscated or encoded script injections.

How It Helps:

- Adds an additional layer of client-side protection.
- Useful in **manual testing or browsing** of suspicious sites.
- Can assist security analysts during **threat hunting or forensics**.

Limitations:

- Not a replacement for server-side or network-level protection.
- May produce false alerts on legitimate websites using complex JS.
- Needs regular updates to adapt to new attack signatures.

☒ Summary

Topic	Key Takeaway
SAST	Scans code for security issues before execution . Best for early detection.
DAST	Scans running applications for vulnerabilities. Best for detecting runtime issues .
Web Framework Security	Use built-in security features responsibly and avoid misconfiguration.
browser-jsguard	A proactive browser tool for spotting client-side malicious activities in real-time.